

Conteúdo da P1

Criação de esquema de Banco de Dados

Criação e modificação de tabelas

create table → Define a estrutura da tabela, suas restrições de integridade e cria uma tabela vazia.

```
CREATE TABLE alunos(  
    id INT NOT NULL UNIQUE, -- Cada aluno tem um número único  
    nome VARCHAR(100) NOT NULL, -- Nome do aluno  
    nascimento DATE, -- Data de nascimento  
    PRIMARY KEY (id) -- Campo id é a chave primária  
);
```

UNIQUE → O valor não pode ser repetir.

DATE → Data no formato ("YYYY-MM-DD").

NOT NULL → Campo obrigatório.

```
CREATE TABLE matriculas(  
    id_matricula INT NOT NULL UNIQUE, -- id único da matrícula  
    id_aluno INT NOT NULL, -- Aluno que fez a matrícula  
    data_matricula DATETIME, -- Data e hora da matrícula  
    PRIMARY KEY (id_matricula),  
    FOREIGN KEY (id_aluno) REFERENCES alunos(id) -- id_aluno deve existir em 'alunos'  
);
```

DATETIME → Formato de data + hora → ("YYYY-MM-DD HH:MM:SS").

FOREIGN KEY → Cria uma ligação entre tabelas. Aqui, id_aluno da tabela de matriculas deve existir como id na tabela alunos. Isso garante consistência no banco.

alter table → Modifica estruturas de uma tabela já existente, como adicionar, remover ou modificar colunas.

```
ALTER TABLE nome_tabela  
    -- ADICIONAR coluna  
    ADD nome_coluna tipo  
  
    -- MODIFICAR coluna existente  
    MODIFY nome_coluna novo_tipo  
  
    -- REMOVER coluna  
    DROP COLUMN nome_coluna  
  
    -- ADICIONAR restrição (ex: chave estrangeira)  
    ADD CONSTRAINT nome_restricao FOREIGN KEY (...) REFERENCES outra_tabela
```

```
-- REMOVER restrição  
DROP CONSTRAINT nome_restricao
```

Exemplos

```
-- Adicionando a coluna nome na tabela Ambulatorios  
ALTER TABLE Ambulatorios  
ADD nome VARCHAR(30)  
  
-- Remove a chave primária da tabela Médicos  
ALTER TABLE Medicos DROP PRIMARY KEY  
  
-- Remove as colunas doenca e cidade da tabela pacientes  
ALTER TABLE Pacientes DROP COLUMN doenca, DROP COLUMN cidade  
  
-- Diz que a coluna nroa na tabela Funcionarios tem que existir  
-- na tabela Ambulatorios  
ALTER TABLE Funcionarios  
ADD FOREIGN KEY(nroa) REFERENCES Ambulatorios  
  
-- Mesmo que o exemplo anterior, porém mas aqui a restrição  
-- ganha um nome  
ALTER TABLE Funcionarios  
ADD CONSTRAINT fk_nroa  
FOREIGN KEY(nroa) REFERENCES Ambulatorios
```

drop table → Remove uma tabela com todas as suas tuplas.

Exemplos de criação de tabela

```
CREAT TABLE ambulatorios(  
    nroa int,  
    andar numeric(3) NOT NULL,  
    capacidade smallint,  
    PRIMARY KEY (nroa)  
);
```

Numeric → Número com até 3 dígitos.

Smallint → Número inteiro pequeno.

```
CREAT TABLE medicos(  
    codm int,  
    nome varchar (40) NOT NULL,  
    idade smallint NOT NULL,  
    especialidade char(20),  
    CPF numeric (11) UNIQUE,  
    cidade varchar(30),
```

```
nroa int,  
PRIMARY KEY(condm),  
FOREIGN KEY (nroa) REFERENCES Ambulatorios  
);
```

Índices

- Definidos sobre atributos para acelerar consultas a dados.
 - Índices são definidos automaticamente para chaves primárias.

INDEX → Cria um índice normal, só para melhorar o desempenho da busca.

UNIQUE INDEX → Cria um índice que não permite valores repetidos.

```
CREATE [UNIQUE] INDEX nome_indice  
ON nome_tabela (Coluna1, coluna2, ...)  
  
-- UNIQUE → Garante que os valores nas colunas não se repitam  
-- nome_indice → nome que você dá ao índice  
-- nome_tabela → onde será criado o índice  
-- (coluna1, ...) → em quais colunas o índice será aplicado
```

```
DROP INDEX nome_indice ON nome_tabela
```

Exemplos

```
CREATE UNIQUE INDEX indPac_CPF ON Pacientes (CPF)
```

- Cria um índice único para a coluna CPF na tabela Pacientes.

```
DROP INDEX indPac_CPF ON Pacientes
```

- Remove o índice indPac_CPF da tabela Pacientes.

Inserção de dados

INSERT → Inserção de dados

- Usado para adicionar registro (linhas) em uma tabela existente

```
INSERT INTO nome_da_tabela  
VALUES (valor1, valor2, valor3, ...)
```

Exemplo

```
INSERT INTO alunos  
VALUES (123, 'Mariana', 'CC', 2022);
```

```
INSERT INTO Medicos
(codm, nome, idade, especialidde, CPF, cidade)
VALUES (4, 'Carlos', 28, 'ortopedia', 111123123, 'Joinville');
```

UPDATE → Atualizar dados existentes

```
UPDATE nome_da_tabela
SET coluna1 = valor1, coluna2 = valor2, ...
WHERE condicao
```

Exemplos

```
UPDATE Medicos
SET cidade = 'Florianopolis'
-- Nesse caso ele muda todas as linhas da tabela
```

```
UPDATE Ambulatorios
SET capacidade = capacidade + 5, andar = 3
WHERE nroa = 2
```

DELETE → Remover dados

```
DELETE FROM nome_da_tabela
WHERE condicao
```

Exemplos

```
DELETE FROM Ambulatorios
WHERE especialidade = 'cardiologia' or cidade <> 'Florianopolis'
```

<> → Diferente em SQL

Consultas básicas

Comando SELECT

- Duplicatas não são eliminadas automaticamente
 - Ao usar **SELECT**, o SQL retorna todas as linhas, mesmo que sejam iguais
 - Para eliminar duplicatas, é preciso usar **DISTINCT**

Funções de agregação

- **COUNT ()** → Conta quantos valores existem
- **MAX () / MIN ()** → Mostra o maior e o menor valor
- **SUM ()** → Soma todos s valores
- **AVG ()** → Faz a média dos valores

Comando SELECT → Eliminação de duplicatas

DISTINCT → Utilizado para eliminação de duplicatas

Exemplo → Buscar as especialidades dos médicos

```
SELECT DISTINCT Especialidade FROM Medicos
```

- Sem **DISTINCT** → Usar quando você quer ver todos os registros reais
- Com **DISTINCT** → usar quando você quer saber quais são as especialidades disponíveis, sem repetir

Comando SELECT → Retorno de valores calculados

Exemplo → Quantos grupos de 5 leitos podem ser formados em cada ambulatória

```
SELECT nroa, capacidade/5 as grupos5 FROM Ambulatorios
```

as grupos5 → Dá o nome “grupos5” para a coluna calculada que aparecerá na no retorno dessa expressão

Exemplo → Qual o salário líquido dos funcionários (descontando 10%)?

```
SELECT CPF, salario - (salario * 0.1) as líquido FROM Funcionarios
```

Comando SELECT → Função COUNT

Exemplo → Informar o total de médicos ortopedistas

```
SELECT count(*) as totalOrtopedistas FROM Medicos WHERE especialidade = 'ortopedia'
```

- **COUNT(*)** → Conta todas as linhas da tabela Médicos que têm especialidade ortopedia
- ***** → Conta a linha inteira, independentemente de haver valores nulos em alguma coluna

Exemplo → Total de médicos que atendem em ambulatórios

```
SELECT count(nroa) as total FROM Medicos
```

- Está contando quantos médicos têm um valor preenchido (não nulo) na coluna nroa
 - Se um médico não atende em ambulatório, ele não é contado

Comando SELECT → Função SUM

Exemplo → Informar a capacidade total dos ambulatórios do primeiro andar

```
SELECT sum(capacidade) as totalAndar1 FROM ambulatorios WHERE andar = 1
```

Comando SELECT → Função AVG

Exemplo → Informar a média de idade dos pacientes de Florianópolis

```
SELECT avg(idade) as mediaPacFpolis FROM Pacientes WHERE cidade = 'Florianópolis'
```

Comando SELECT → Funções MAX / MIN

Exemplo → Informar o menor e o maior salário pagos aos Funcionários do departamento pessoal com mais de 50 anos

```
SELECT min (salario) as minimo, max(salario) as maximo  
FROM Funcionarios  
WHERE dept = 'Pessoal' and idade > 50
```

Comando SELECT → Funções de agregação com Distinct

- Valores duplicados não são computados

Exemplos

```
SELECT count (distinct especialidade) FROM Medicos  
-- Conta quantas especialidades diferentes existem na tabela Medicos
```

```
SELECT avg(distinct salario) FROM Funcionarios  
-- Calcula a media dos salarios unicos da tabela Funcionarios  
-- Se varios funcionarios ganham o mesmo salario, esses valores  
-- duplicados sao ignorados no calculo da media
```

Cláusula WHERE

[NOT] LIKE → Busca por padrões

% → Casa com qualquer sequência de caracteres (até nenhuma letra)

Exemplo → Buscar CPF e nome dos médicos com inicial M

```
SELECT CPF, nome FROM Medicos WHERE nome like 'M%'
```

Exemplo → Buscar nomes de pacientes cujo CPF termina com 20000 ou 30000

```
SELECT nome FROM Pacientes WHERE CPF like '%20000' or CPF like '%30000'
```

- Em Alguns dialetos SQL, '*' é usado ao invés de '%'

_ → Casa com exatamente um caractere

[a-f] → Casa com qualquer caractere entre a e f (apenas no SQL Server)

Exemplos

```
-- nomes que terminam com "a"  
where nome like '%a'
```

```
-- nomes que têm "ar" no meio
where nome like '%ar%'

-- nomes com 4 letras e a segunda letra sendo "a"
where nome like '_a__'
```

IS [NOT] NULL → Teste de existência de valores nulos

Exemplo → Buscar CPF e o nome dos médicos que não dão atendimento em ambulatórios

```
SELECT CPF, nome FROM Medicos WHERE nroa is NULL
```

[NOT] BETWEEN valor1 AND valor2 → Busca por intervalos de valores

Exemplo → Buscar os dados das consultas marcadas para o período da tarde

```
SELECT * FROM Consultas WHERE hora BETWEEN '14:00' AND '18:00'
```

[NOT] IN → Teste de pertinência elemento-conjunto

Exemplo → Buscar os dados dos médicos ortopedistas, traumatologistas e cardiologistas de Florianópolis

Junções

JOIN ou INNER JOIN

```
SELECT * FROM Alunos JOIN Turmas ON Alunos.turma = Turmas.turma;
```

- Junta apenas onde há correspondência entre **Alunos.turma** e **Turmas.turma**
- Equivale a um "interseção"

id_aluno	nome	turma	sala
1	Ana	A	101
2	Bruno	B	102

- Clara (turma C) e turma D **ficam de fora** porque não têm par

NATURAL JOIN

```
SELECT * FROM alunos NATURAL JOIN Turmas;
```

- Junta automaticamente pelas colunas com mesmo nome
- Remove colunas duplicadas no resultado
- Equivale a um **INNER JOIN** + **USANDO** os nomes iguais

id_aluno	nome	turma	sala
1	Ana	A	101

2	Bruno	B	102
---	-------	---	-----

- Clara e turma D também não aparecem
- Tome cuidado: ele pode juntar pelas colunas erradas se tiver muitos nomes iguais

LEFT JOIN

```
SELECT * FROM Alunos LEFT JOIN Turmas ON Alunos.turma = Turmas.turma;
```

- Junta mantendo todos os dados da tabela da esquerda (**Alunos**)
- Onde não houver correspondência na tabela da direita, aparece NULL

id_aluno	nome	turma	sala
1	Ana	A	101
2	Bruno	B	102
3	Clara	C	NULL

- Mostra Clara mesmo sem sala

RIGHT JOIN

```
SELECT * FROM Alunos RIGHT JOIN Turmas ON Alunos.turma = Turmas.turma;
```

- Junta mantendo todos os dados da tabela da direita (**Turmas**)
- Se não houver aluno, aparece NULL nos dados de **Alunos**

id_aluno	nome	turma	sala
1	Ana	A	101
2	Bruno	B	102
NULL	NULL	D	104

- Mostra a turma D mesmo sem aluno

Regras para usar as operações de conjuntos

- Mesmo número de colunas e tipos compatíveis
 - Se uma consulta retorna nome (texto) e idade(inteiro), a outra também precisa retornar dois campos com tipos correspondentes
- Mesmo grau → número de colunas
 - Se uma retorna 2 colunas, a outra também precisa retornar 2 colunas

Operações sobre conjuntos

União (UNION)

Exemplo → Buscar o nome e idade de médicos e funcionários


```
SELECT nome, idade FROM medicos
UNION
SELECT nome, idade FROM funcionarios
```

<i>Medicos</i>		<i>Funcionarios</i>		<i>Medicos U Funcionarios</i>	
nome	idade	nome	idade	nome	idade
Ana	45	José	23	Ana	45
Vera	52	Anne	32	Vera	52
Rui	52	Rui	52	Rui	52
				José	23
				Anne	32

Intersecção (INTERSECT)

Exemplo → Buscar o nome e idade de médicos que também são funcionários

```
SELECT nome, idade FROM medicos
INTERSECT
SELECT nome, idade FROM funcionarios
```

<i>Medicos</i>		<i>Funcionarios</i>		<i>Medicos $\cap$ Funcionarios</i>	
nome	idade	nome	idade	nome	idade
Ana	45	José	23		
Vera	52	Anne	32		
Rui	52	Rui	52	Rui	52

Diferença (EXCEPT)

Exemplo → Buscar o nome e idade dos médicos que não são funcionários

```
SELECT nome, idade FROM medicos
EXCEPT
SELECT nome, idade FROM funcionarios
```

<i>Medicos</i>		<i>Funcionarios</i>		<i>Medicos - Funcionarios</i>	
nome	idade	nome	idade	nome	idade
Ana	45	José	23	Ana	45
Vera	52	Anne	32	Vera	52
Rui	52	Rui	52		

Exemplo → Buscar o nome e idade dos médicos mais velhos

```

SELECT nome, idade FROM medicos -- Retorna todos os médicos
EXCEPT
SELECT m1.nome, m1.idade FROM medicos m1 -- Retorna os médicos mais novos
JOIN medicos m2 ON m1.idade < m2.idade -- Aquele que têm pelo menos um
-- um outro médicos mais velho que ele

-- No fim, remove da lista total de médicos aqueles que não são
-- os mais velhos
-- Todos os medicos - os que têm alguém mais velho = os mais velhos

```

Medicos (m1)		Medicos (m2)		m1 JOIN m2 ON m1.idade<m2.idade			
nome	idade	nome	idade	m1.nome	m1.idade	m2.nome	m2.idade
Ana	45	Ana	45	Ana	45	Vera	52
Vera	52	Vera	52	Ana	45	Rui	52
Rui	52	Rui	52				

- Exemplo 2: Buscar nome e idade dos médicos mais velhos:

```

SELECT nome, idade FROM médicos

EXCEPT

```

nome	idade
Ana	45
Vera	52
Rui	52

```

SELECT m1.nome, m1.idade FROM médicos m1 JOIN
médicos m2 ON m1.idade<m2.idade

```

nome	idade
Ana	45

Resultado

nome	idade
Vera	52
Rui	52

- Quando você usa subconsultas, está fazendo consultas dentro de consultas.
 - É uma forma de relacionar dados entre tabelas sem precisar usar `JOIN` diretamente
 - Permite filtrar previamente os dados, antes de comparar com a tabela externa

[NOT] IN

- Verifica se um valor está (ou não está) dentro do resultados da subconsulta

```

SELECT lista_atributos
FROM tabela 1
WHERE atributo_ou_expressao [NOT] IN (Consulta_SQL)

```

Exemplo → Retorna o nome dos médicos que realizaram consultas no dia 06/11/13

```
SELECT nome FROM medicos WHERE codm
IN (SELECT codm FROM consultas WHERE data = '06/11/13')
```

Exemplo → Retorna os CPFs dos funcionários que não são pacientes.

```
SELECT cpf FROM funcionarios WHERE cpf
NOT IN (SELECT cpf FROM pacientes)
```

Exemplo → Retorna os CPFs das pessoas que são ao mesmo tempo médicos e pacientes

```
SELECT cpf FROM medicos WHERE cpf
IN (SELECT cpf FROM pacientes)
```

ANY

- Compara o valor com qualquer um dos resultados da subconsulta
 - A condição será verdadeira se for verdadeira para qualquer valor da subconsulta

```
select lista_atributos
from tabela1 [, ...]
where atributo_ou_expressão
[= | < | <= | > | >= | <> | !=] ANY (consulta_SQL)
```

Exemplo → Retornar os nomes dos médicos que atenderam em 06/11/13.

```
SELECT nome FROM medicos WHERE codm =
ANY (SELECT codm FROM consultas WHERE data = '06/11/13')
```

Exemplo → Pegar os funcionários mais jovens que algum outro funcionário.

```
SELECT nome FROM funcionarios WHERE idade <
ANY (SELECT idade FROM funcionarios)
```

ALL

- Compara o valor com todos os resultados da subconsulta
 - A condição só será verdadeira se for verdadeira para todos os valores da subconsulta

```
select lista_atributos
from tabela1 [, ...]
where atributo_ou_expressão
[= | < | <= | > | >= | <> | !=] ALL (consulta_SQL)
```

Exemplo → Essa consulta retorna os funcionários com o maior salário de toda a empresa, em comparação ao setor contábil.

```
SELECT nome FROM funcionarios
WHERE salario > ALL (SELECT salario FROM funcionarios
WHERE departamento = 'contabil')
```

Exemplo → A consulta retorna os nomes dos funcionários que não são pacientes.

```
SELECT nome FROM funcionarios
WHERE cpf <> all (ou NOT IN) (SELECT cpf FROM pacientes)
```

Comparações elemento-elemento

- É quando você compara um único valor de uma tabela com um único valor vindo de uma subconsulta

Exemplo → Retorna os nomes dos funcionários que ganham mais que o funcionário de CPF **2200020000**

```
SELECT nome FROM funcionarios WHERE salario >
(SELECT salario FROM funcionarios WHERE cpf = '22000210212')
```

Exemplo → Lista o nome e CPF dos médicos, cujo CPF é menor que **10000100001**, e cuja especialidade é igual à do médico com esse CPF.

```
SELECT nome, cpf FROM medicos WHERE cpf < 10121201201 AND
especialidade = (SELECT especialidade FROM medicos WHERE cpf
= 123123000)
```

EXISTS

- Ele testa se um predicado é verdadeiro para pelo menos um elemento do conjunto analisado
 - O **EXISTS** executa uma subconsulta para cada tupla da consulta externa e retorna verdadeiro se a subconsulta retornar pelo menos um resultado

```
SELECT lista_atributos
FROM tabela1
WHERE [NOT] EXISTS (subconsulta_SQL);
```

Exemplo → retorna todos os médicos que realizaram pelo menos uma consulta em '06/11/13'.

```
SELECT nome FROM medicos m WHERE exists
(SELECT * FROM consultas WHERE data = '06/11/13' AND codm = m.codm)
```

Exemplo → retorna os funcionários do departamento 'pessoal' que não são pacientes.

```
SELECT f.nome FROM funcionarios f WHERE f.depto = 'pessoal'
AND NOT EXISTS (SELECT * FROM pacientes WHERE cpf = f.cpf)
```



EXISTS – *Verifica se a subconsulta retorna pelo menos uma linha*

👉 **Funciona como um teste: "Existe algum resultado?"**

Se a subconsulta retornar **pelo menos uma linha**, a condição é **VERDADEIRA**, e a linha da consulta externa é **incluída** no resultado.

✗ **NOT EXISTS** – *Verifica se a subconsulta não retorna nenhuma linha*

👉 **Funciona como um teste: "Não existe nenhum resultado?"**

Se a subconsulta **não retornar nenhuma linha**, a condição é **VERDADEIRA**, e a linha da consulta externa é **incluída**.

Subconsultas, Ordenação e Agrupamento (1)

✓ **EXISTS** – *Verifica se a subconsulta retorna pelo menos uma linha*

👉 **Funciona como um teste: "Existe algum resultado?"**

Se a subconsulta retornar **pelo menos uma linha**, a condição é **VERDADEIRA**, e a linha da consulta externa é **incluída** no resultado.

✗ **NOT EXISTS** – *Verifica se a subconsulta não retorna nenhuma linha*

👉 **Funciona como um teste: "Não existe nenhum resultado?"**

Se a subconsulta **não retornar nenhuma linha**, a condição é **VERDADEIRA**, e a linha da consulta externa é **incluída**.

Operador EXISTS

Exemplo → Neste exemplo estamos retornando nomes de animais da espécie Hipopótamo

```
SELECT nomeanimal FROM animais a WHERE  
EXISTS (SELECT * FROM especies e WHERE e.codespecie = a.codespecie  
AND e.nome = 'Hipopótamo')
```

Operador NOT EXISTS

Exemplo 3:

```
SELECT nomeespecie FROM especies e1
WHERE
  NOT EXISTS (Select * From especies e2
              Where e2.expectativavida>e1.expectativavida);
```

- Neste exemplo, para cada espécie verifica se NÃO EXISTE uma outra espécie com expectativa maior que a dele:
 - Para **e1.codespecie=1** a subconsulta não acha um **e2** com expectativa > 10. Logo, como a subconsulta não retorna nada (0=FALSE), o NOT do (NOT EXISTS) aplicará !(FALSE) = TRUE. Isto é, fará **e1** (Hipopótamo) retornar.

e1->

cod-espécie	nome-especie	expectativa-vida
1	Hipopótamo	10
e2-> 2	Coelho	3

Resultado:

nomeespecie
Hipopótamo

Exemplo 3:

```
SELECT nomeespecie FROM especies e1
WHERE
  NOT EXISTS (Select * From especies e2
              Where e2.expectativavida>e1.expectativavida);
```

- Neste exemplo, para cada espécie verifica se NÃO EXISTE uma outra espécie com expectativa maior que a dele:
 - Para **e1.codespecie=2** a subconsulta acha um **e2** com expectativa > 3. Logo, como a subconsulta retorna algo (>0=TRUE), o NOT do (NOT EXISTS) aplicará !(TRUE) = FALSE. Isto é, fará **e1** (Coelho) não retornar.

e2->

cod-espécie	nome-especie	expectativa-vida
1	Hipopótamo	10
e1-> 2	Coelho	3

Resultado:

nomeespecie
Hipopótamo

Exemplo 4: Encadeando múltiplos NOT EXISTS

```
SELECT nome FROM funcionarios f WHERE
  NOT EXISTS (Select * From animais a Where
              NOT EXISTS (Select * From consultas c Where
                          c.matricula=f.matricula And c.codanimal=a.codanimal))
```

f->

matricula	nome
111	Vera
222	Lúcia
333	Rita

cod-animal	nome-animal
1	Glória
2	Vermelho
3	Jaja

cod-animal	matricula
1	111
2	111
1	222
3	333
3	111
2	333

Resultado:

nome
Vera

Subconsulta na Cláusula FROM

- Gera uma **tabela derivada** a partir de uma ou mais tabelas, para uso na consulta externa

- *otimização*: filtra linhas e colunas de uma tabela que são desejadas pela consulta externa

```
select lista_atributos  
from (consulta_SQL) as nome_tabela_derivada
```

Exemplo → Médicos que consultaram em 06/11/2013

```
SELECT medicos.*, C.hora FROM medicos JOIN  
(SELECT codm, hora FROM consultas WHERE = '06/11/13') ON  
medicos.codm = c.codm
```

Exemplo → Ambulatórios com médicos de Florianópolis

```
SELECT amb.* FROM (SELECT nroa, andar FROM ambulatorios) as amb  
JOIN (SELECT nroa FROM medicos WHERE cidade = 'Fpolis') as mflo  
ON amb.nroa = mflo.nroa
```

Atualização com Consultas

- Comandos de atualização podem incluir comando de consulta
 - Necessário toda vez que a atualização deve testar relacionamento entre tabelas

Exemplo

```
DELETE FROM consultas WHERE hora > '17:00:00'  
AND codm IN (SELECT codm FROM medicos WHERE nome = 'Maria')
```

Exemplo

```
UPDATE medicos SET nroa = NULL WHERE NOT EXISTS  
(SELECT * FROM medicos m WHERE m.codm <> medicos.codm AND  
m.nroa = medicos.nroa)
```

Exemplo

```
UPDATE ambulatorios set capacidade = capacidade +  
(SELECT capacidade FROM ambulatorios WHERE nroa = 4) WHERE nroa = 2
```

Exemplo

```
INSERT INTO medicosNovos SELECT codm, nome, especialidade FROM medicos WHERE id  
ade < 21;
```

Ordenação de Resultados

Cláusula → **ORDER BY**

```
select lista_atributos
from lista_tabelas
[where condição]
[order by nome_atributo_1 [desc] [, nome_atributo_n [desc]]]
```

Exemplo

```
SELECT * FROM pacientes order by nome
```

Exemplo

```
SELECT salario, nome FROM funcionarios ORDER BY salario DESC, nome
```

- É possível determinar a quantidade de valores ordenados a retornar

```
SELECT * FROM pacientes ORDER BY nome LIMIT 5
```

```
SELECT salario, nome FROM funcionarios ORDER BY
salario DESC, nome LIMIT 5,10
```

Definição de Grupos

Cláusula → **GROUP BY**

```
select lista_atributos
from lista_tabelas
[where condição]
[group by lista_atributos_agrupamento
[having condição_para_agrupamento]]
```

Exemplo

```
SELECT especialidade, count(*) FROM medicos GROUP BY especialidade
```


• Exemplo

```
select especialidade, count(*)
from Médicos
group by especialidade
```



especialidade	Count
ortopedia	2
pediatria	1
neurologia	1
traumatologia	3

especialidade	"grupos"					
ortopedia	codm	nome	idade	RG	cidade	nroa
	1	João	40	1000010000	Fpolis	1
	4	Carlos	28	1100011000	Joinville	
pediatria	codm	nome	idade	RG	cidade	nroa
	3	Pedro	51	1100010000	Fpolis	2
neurologia	codm	nome	idade	RG	cidade	nroa
	5	Márcia	33	1100011100	Biguaçu	3
traumatologia	codm	nome	idade	RG	cidade	nroa
	2	Maria	42	1000011000	Blumenau	2
	6	Joana	37	1111110000	Fpolis	3
	7	Mauro	53	1111000011	Blumenau	2

Cláusula → **HAVING**

- Define condições para que grupos sejam formados
 - Condições só podem ser definidas sobre atributos do agrupamento ou serem funções de agregação
 - Existe somente associada à cláusula **GROUP BY**

```
SELECT especialidade, count(*) FROM medicos
GROUP BY especialidade HAVING count(*) > 1
```



Dica de ouro pra lembrar:

Essa estrutura:

```
sql
CopiarEditar
WHERE NOT EXISTS (
  SELECT * FROM X
  WHERE NOT EXISTS (
    SELECT * FROM Y
  )
)
```

É uma forma de dizer:

“Eu quero aqueles para quem todos os X têm um Y correspondente.”

O que é uma Visão (View)?

- A visão é uma tabela “virtual” cujo conteúdo é derivado de outras tabelas base
 - Ela não existe fisicamente, por isso as operações de atualização são limitadas

- Operações de consulta são totalmente permitidas

Criação e utilização de Visões

Exemplo de criação

```
CREATE OR REPLACE VIEW NOME_VIEW (ATRIBUTOS) AS
// COMANDO SELECT

// EXEMPLO CONCRETO
// Atributos só são adicionados caso eu queira modificar o nome da coluna da nova
// tabela que vai ser criada pela visão
CREATE VIEW TRABALHO_EM1 (PNOME, UNOME, PJNOME, HORAS) AS
  SELECT PNOME, UNOME, PJNOME, HORAS
  FROM EMPREGADO, PROJETO, TRABALHA_EM
  WHERE SSN = ESSN AND PNO=PNUMERO
  ORDER BY NOME;
```

Exemplo de consulta → As consultas são feitas da mesma forma que em tabelas normais

```
SELECT PNOME, UNOME
FROM TRABALHA_EM1
WHERE PJNOME = 'ProjetoX'
```

Exclusão de uma visão → `DROP VIEW`

Atualização de visões

- As regras para atualizar uma visão (com `INSERT`, `UPDATE`, `DELETE`) são
 - Visões de tabela única → São atualizáveis
 - Visões com junções → Não são atualizáveis
 - Visões com agregação → Não são atualizáveis

`WITH CHECK OPTION` → Pode ser adicionada na definição da visão para que o SGBD verifique se as atualizações são possíveis

Estrutura básica de uma função

```
CREATE OR REPLACE FUNCTION nome_funcao(parametros) RETURNS tipo AS
$$
[DECLARE]
  -- Declaração de variáveis
BEGIN
  -- Corpo da função (comandos)
END;
```

```
$$  
LANGUAGE plpgsql;
```

CREATE FUNCTION → Criar uma nova função

CREATE OR REPLACE FUNCTION → Criar ou atualizar um função

DROP FUNCTION nome_funcao(tipo_dos_parametros) → Apagar função

Sobrecarga → Permite criar múltiplas funções com o mesmo nome, desde que a assinatura (os parâmetros) seja diferente

Variáveis e tipos de dados

Declaração → Variáveis locais são declaradas na seção **DECLARE**

Sintaxe → nome_variavel [CONSTANT] tipo [NOT NULL] [DEFAULT valor];

- Ao utilizar o **CONSTANT** é proibido que o valor da variável seja alterado e é obrigatório definir valor na declaração da variável
 - O mesmo ocorre com o **NOT NULL** em relação a declaração da variável

Tipos especiais

- %TYPE** : A variável assume o mesmo tipo de uma coluna de uma tabela (cliente.nome%TYPE).
- %ROWTYPE** : A variável assume a estrutura de uma linha inteira de uma tabela (cliente%ROWTYPE).
- RECORD** : Semelhante ao **%ROWTYPE** , mas pode armazenar uma linha de qualquer estrutura, não estando atrelado a uma tabela específica.

Exemplos

- url varchar := ' http://mysite.com ';
- user_id CONSTANT integer := 10;
- weight integer[] DEFAULT '{10,9,8,7,6,5,4,3,2,1}';
- helpstr varchar(300);

Estruturas de Controle

Condicional → IF ... THEN ... ELSIF ... THEN ... ELSE ... END IF;

Laços de Repetição (Loops)

- FOR** → Itera sobre uma sequência de números (FOR i IN 1..10 LOOP ... END LOOP;) ou sobre os resultados de uma consulta (FOR registro IN SELECT * FROM tabela LOOP).

```
FOR count IN REVERSE 10..1 BY 2 LOOP  
  -- count assumirá os valores 10,8,6,4,2  
END LOOP;
```

- WHILE ... LOOP ... END LOOP ...;** → Executa enquanto uma condição booleana for verdadeira.
- LOOP ... END LOOP;** → Cria um laço infinito que precisa de uma condição de saída explícita (EXIT ou RETURN).

```

LOOP
  -- executa alguma coisa
  IF count > 0 THEN
    EXIT;
  END IF;
END LOOP;

```

```

LOOP
  -- executa alguma coisa
  EXIT WHEN count > 100;
  CONTINUE WHEN count > 50;
  -- executa esse trecho quando 50 <= count <= 100
END LOOP;

```

Controle de Laços:

- **EXIT** → Interrompe a execução do laço.
- **CONTINUE** → Pula para a próxima iteração do laço.

Consultas SQL

- Comandos DML (SELECT, INSERT, etc.) e DDL (CREATE TABLE, etc.) podem ser executados diretamente no corpo da função
- **INSERT INTO** → Para armazenar o resultado de uma consulta em uma variável
- **GET DIAGNOSTICS** → Usado para obter informações sobre a última operação, como o número de linhas afetadas (**ROW_COUNT**)

```

CREATE FUNCTION limpa_emp() RETURNS integer AS
$$
DECLARE
  linhas_afetadas integer DEFAULT 0;
BEGIN
  DELETE FROM emp WHERE salario < 0;
  GET DIAGNOSTICS linhas_afetadas = ROW_COUNT;
  RETURN linhas_afetadas;
END;
$$
LANGUAGE
plpgsql;

```

- SQL dinâmicas mais complexas

```

CREATE FUNCTION apagaReg(tabela text, chave text, id int4) RETURNS int4 AS

```

```

$$
DECLARE vLinhas int4 DEFAULT 0;
BEGIN
    EXECUTE 'DELETE FROM ' || tabela || ' WHERE ' || chave || ' = ' || id;
    GET DIAGNOSTICS vLinhas = ROW_COUNT;
    RETURN vLinhas;
END;
$$
LANGUAGE 'plpgsql';

```

O que são gatilhos

- É uma especificação de ação que o banco de dados deve executar toda vez que determinado evento ocorrer

Componentes e tipos de gatilhos

Evento → A operação que dispara o gatilho. Pode ser `INSERT`, `UPDATE`, `DELETE` ou `TRUNCATE`

Momento → Quando a função do gatilho deve ser executada

- `BEFORE`
- `AFTER`
- `INSTEAD OF` → Usado para visões

Nível

- `FOR EACH ROW` → A função é executada uma vez para cada linha afetada pela operação
- `FOR EACH STATEMENT` → A função é executada uma vez por comando SQL

Criação de um Gatilho

```

CREATE TRIGGER nome_do_gatilho
{ BEFORE | AFTER | INSTEAD OF } { evento [OR ...] }
ON nome_da_tabela_ou_view
[ FOR [EACH] { ROW | STATEMENT } ]
EXECUTE PROCEDURE nome_da_funcao (argumentos);

```

- Apagar → `DROP TRIGGER [IF EXISTS] nome_do_gatilho ON tabela;`
- Renomear → `ALTER TRIGGER nome_do_gatilho ON tabela RENAME TO novo_nome;`

Funções para Gatilhos

```

CREATE FUNCTION primeira_funcao() RETURNS trigger AS
$$
BEGIN
    RETURN NULL;
END;

```

```
$$  
LANGUAGE plpgsql;
```

Variáveis especiais

- **NEW** (RECORD) → Contém a nova linha de dados em operações de **INSERT** e **UPDATE**. É nulo (**NULL**) em gatilhos **STATEMENT**.
- **OLD** (RECORD) → Contém a linha antiga (original) de dados em operações de **UPDATE** e **DELETE**.
- **TG_OP** (TEXT): A operação que disparou o gatilho: 'INSERT', 'UPDATE' ou 'DELETE'.
- **TG_WHEN** (TEXT): O momento do disparo: 'BEFORE', 'AFTER' ou 'INSTEAD OF'.
- **TG_LEVEL** (TEXT): O nível de granularidade: 'ROW' ou 'STATEMENT'.
- **TG_TABLE_NAME** (NAME): O nome da tabela onde o gatilho foi disparado.
- **TG_NARGS** e **TG_ARGV[]**: O número de argumentos e um array com os argumentos passados no comando **CREATE TRIGGER**.

Exemplo

```
CREATE FUNCTION emp_stamp() RETURNS trigger AS  
$$  
BEGIN  
    IF NEW.empname IS NULL THEN  
        RAISE EXCEPTION 'empname cannot be null';  
    END IF;  
  
    IF NEW.salary IS NULL OR NEW.salary < 0 THEN  
        RAISE EXCEPTION 'salary cannot be null or < 0';  
    END IF;  
  
    NEW.last_date := current_timestamp;  
    NEW.last_user := current_user;  
    RETURN NEW;  
END;  
$$  
LANGUAGE plpgsql;  
  
CREATE TRIGGER emp_stamp BEFORE INSERT OR UPDATE ON emp  
FOR EACH ROW EXECUTE PROCEDURE emp_stamp();
```

Tratamento de erros

- **RAISE NOTICE** → Enviar uma mensagem informativa e continua a execução
- **RAISE EXCEPTION** → Gera um erro, aborta a função e reverte a transação atual

O que é uma Transação

- É uma unidade lógica de processamento no banco de dados, que agrupa um ou mais comandos SQL
- Uma transação é delimitada por um início e fim
 - `BEGIN TRANSACTION;`
 - `END TRANSACTION;`

Operações Básicas

- `read_item(X)` : Localiza o bloco de dados no disco, copia-o para um buffer na memória e, em seguida, copia o item para a variável do programa.
- `write_item(X)` : Localiza o bloco no disco, copia-o para a memória, atualiza o item no buffer com o valor da variável do programa e, por fim, armazena o bloco atualizado de volta no disco

Problemas de concorrência

Quando múltiplas transações são executadas simultaneamente, podem ocorrer os seguintes problemas:

- **Atualização Perdida (Lost Update):** Ocorre quando duas transações acessam e alteram o mesmo item, e a atualização de uma delas é sobrescrita (perdida) pela outra, resultando em um valor incorreto.
- **Atualização Temporária (Leitura Suja / Dirty Read):** Ocorre quando uma transação lê um valor alterado por outra transação que ainda não foi efetivada (comitada). Se a segunda transação falhar e for revertida, a primeira transação terá lido um valor "sujo" que nunca existiu de fato no banco de dados.
- **Sumário Incorreto:** Acontece quando uma transação de agregação (como `SUM`) lê alguns valores antes de serem atualizados por outra transação e outros valores depois, levando a um resultado inconsistente.

Propriedades ACID

Para garantir a integridade dos dados, toda transação deve seguir quatro propriedades fundamentais, conhecidas como ACID:

Atomicidade (Atomicity): Todas as operações da transação devem ser efetivadas com sucesso no BD ou nenhuma das operações será efetivada

Consistência (Consistency): Uma transação sempre deve conduzir o BD de um estado consistente para outro estado também consistente;

Isolamento (Isolation): Em um conjunto de transações concorrentes, cada transação deve ser executada no BD como se ela fosse a única

Durabilidade (Durability): O BD deve garantir que as modificações realizadas por uma transação que concluiu com sucesso sejam persistidas.

Níveis de Isolamento em SQL

Nível de Isolamento	Leitura Suja	Leitura Não Repetível	Fantasmas	Observações
<code>READ UNCOMMITTED</code>	Sim	Sim	Sim	Permite todos os problemas. Não é implementado no PostgreSQL28.

READ COMMITTED	Não	Sim	Sim	
REPEATABLE READ	Não	Não	Sim	Garante que a mesma leitura dentro da transação retorne sempre os mesmos dados, mas pode sofrer com "fantasmas" (novas linhas inseridas por outra transação) ³⁷³⁷³⁷³⁷ .
SERIALIZABLE	Não	Não	Não	Nível mais rigoroso ⁴¹ . Simula a execução das transações em série, uma após a outra, eliminando todas as anomalias ⁴² .

- **Leitura Não Repetível:** Ocorre quando uma transação lê o mesmo dado duas vezes e obtém resultados diferentes porque outra transação o alterou entre as leituras.
- **Fantasmas:** Ocorre quando uma transação executa uma consulta baseada em uma condição (ex: `WHERE`), e outra transação insere novas linhas que satisfazem essa condição, fazendo com que a primeira transação veja linhas "fantasmas" se repetir a consulta

Comandos de controle de transação

- `COMMIT` : Confirma a transação atual, tornando todas as suas alterações permanentes e visíveis para outras transações.
- `ROLLBACK` : Reverte (desfaz) todas as alterações realizadas na transação atual.
- `SAVEPOINT` : Cria um "ponto de salvamento" dentro de uma transação. O comando `ROLLBACK TO savepoint_name` permite reverter a transação apenas até aquele ponto, descartando seletivamente parte das operações sem anular a transação inteira.
- É um mecanismo essencial dos SGBDs para garantir a atomicidade e durabilidade das transações
 - O seu objetivo é levar o banco de dados de um estado inconsistente, causado por uma falha, para um estado consistente anterior

Ações básica de recuperação

- UNDO → Desfazer uma atualização no BD
- REDO → Refazer uma atualização no BD

Tipos de falhas

Tipo de Falha	Causa Comum	Impacto	Ação de Recuperação
Falha de Transação	Erro de lógica, deadlock, cancelamento pelo usuário ⁷ .	A transação ativa é encerrada anormalmente. Não afeta memória principal ou disco.	Transaction UNDO
Falha de Sistema	Queda de energia, falha no S.O. ou hardware.	O SGBD encerra sua execução. A memória principal é perdida, mas o disco não é comprometido.	Global UNDO e PARTIAL REDO
Falha de Meio de Armazenamento	Setores de disco corrompidos, falha no cabeçote de leitura/gravação.	O BD se torna total ou parcialmente inacessível. O disco é comprometido.	Global REDO

Transaction UNDO → Uma transação não concluiu suas operações

- As modificações realizadas por esta transação no BD são desfeitas

Global UNDO → Uma ou mais transações não concluíram as suas operações

- As modificações realizadas por todas estas transações no BD são desfeitas

Partial REDO → Na ocorrência de uma falha, algumas transações podem ter concluído suas operações (committed), mas suas ações podem não ter se refletido no BD

- As modificações realizadas por estas transações são refeitas no BD

Global REDO → No caso de um comprometimento do BD, todas as transações committed no BD são perdidas

- As modificações realizadas por todas estas transações no BD são refeitas

Técnicas de recuperação baseadas em Log

- Esta é a abordagem mais comum e utiliza um arquivo de **Log** (ou *Journal*) para registrar sequencialmente todas as atualizações feitas pelas transações. O log é consultado para realizar operações **UNDO** e **REDO**
 - tipos de log
 - log de UNDO – mantém apenas o valor antigo do dado (before image)
 - log de REDO – mantém apenas o valor atualizado do dado (after image)
 - log de UNDO/REDO (mais comum) – mantém os valores antigo e atualizado do dado

– início de transação: <start T_x >

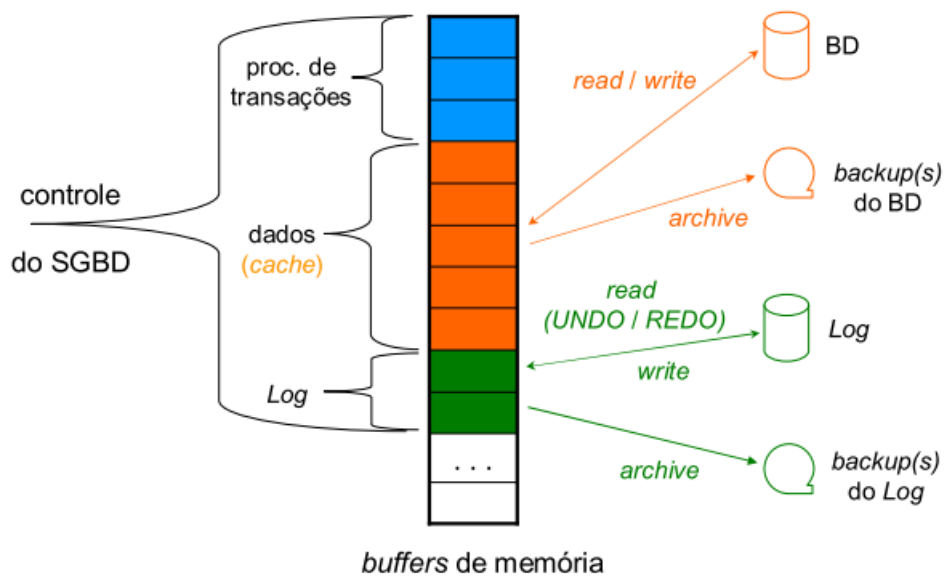
– *commit* de transação: <commit T_x >

– atualização: <write T_x , X, beforeImage, afterImage>

não é necessário em log REDO ←

não é necessário em log UNDO ←

Gerenciamento do Buffer



- **Princípio Write-Ahead-Log (WAL):** Antes de um bloco de dados ser modificado no disco, o registro de log correspondente a essa modificação **deve** ser gravado no disco primeiro.

Gerenciamento de Buffer e sua influência na recuperação

- **STEAL:** Permite que um bloco modificado na cache por uma transação ainda não concluída seja gravado no disco. Isso requer a necessidade de **UNDO** em caso de falha.
- **NOT-STEAL:** Proíbe que um bloco modificado seja gravado no disco antes do commit da transação. Simplifica a recuperação, pois elimina a necessidade de **UNDO**.
- **FORCE:** Força a gravação no disco de todos os blocos modificados por uma transação assim que ela realiza o commit. Garante a durabilidade, mas pode ser ineficiente.
- **NOT-FORCE:** Não força a gravação imediata dos blocos após o commit. Melhora o desempenho, mas exige a necessidade de **REDO** em caso de falha.

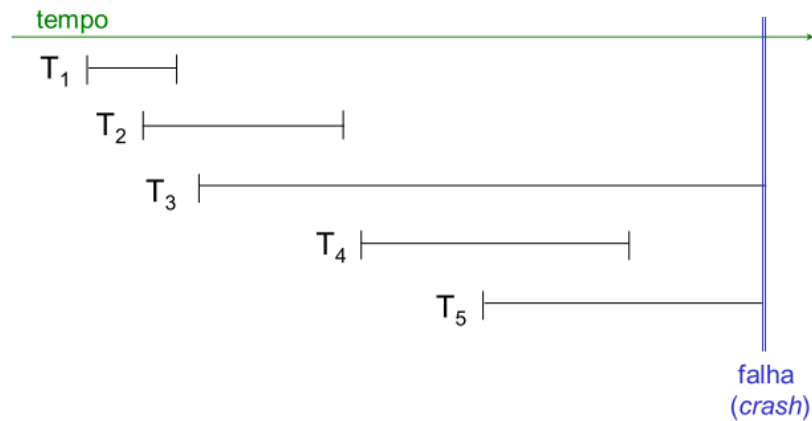
Algoritmos de recuperação baseados em Log

Modificação Imediata do BD (STEAL / NOT-FORCE)

- As atualizações podem ser gravadas no disco antes do commit da transação.

Técnica UNDO/REDO: É a mais comum.

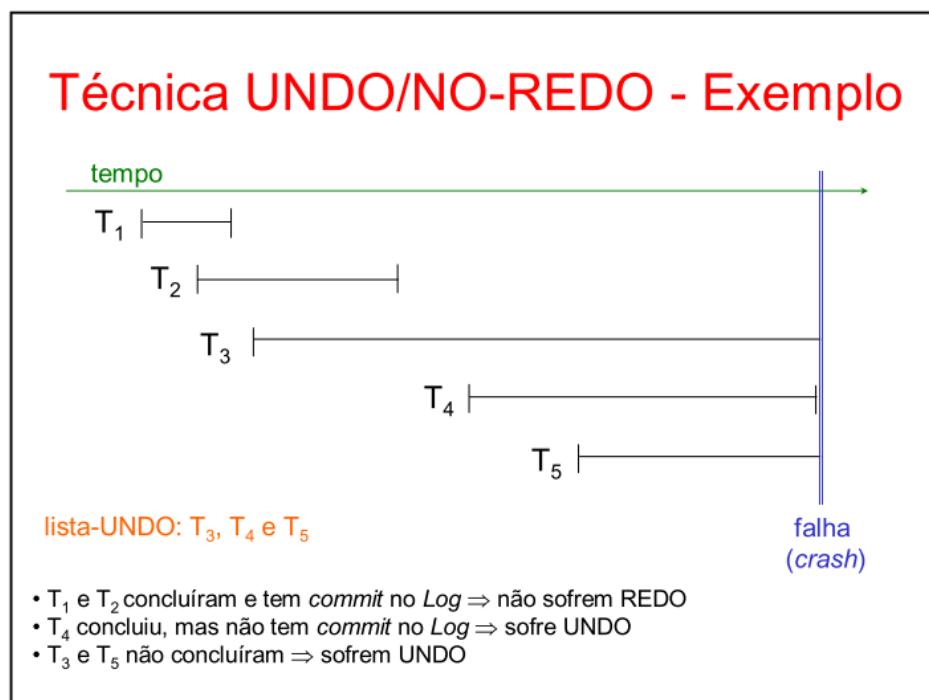
- **Procedimento:** Após uma falha, transações que iniciaram mas não possuem um **<commit>** no log devem sofrer **UNDO**. Transações que possuem um **<commit>** no log devem sofrer **REDO** para garantir que suas alterações estejam no disco.
- O processo geralmente envolve uma varredura para trás (*backward*) do log para **UNDO** e uma para frente (*forward*) para **REDO**.



lista-UNDO: T₃, T₅ (devem sofrer UNDO)

lista-REDO: T₁, T₂, T₄ (devem sofrer REDO)

- **Técnica UNDO/NO-REDO:** Usa a política **FORCE**.
 - **Procedimento:** Como as alterações são forçadas no disco no momento do commit, não há necessidade de **REDO**. Apenas as transações não concluídas precisam sofrer **UNDO**.

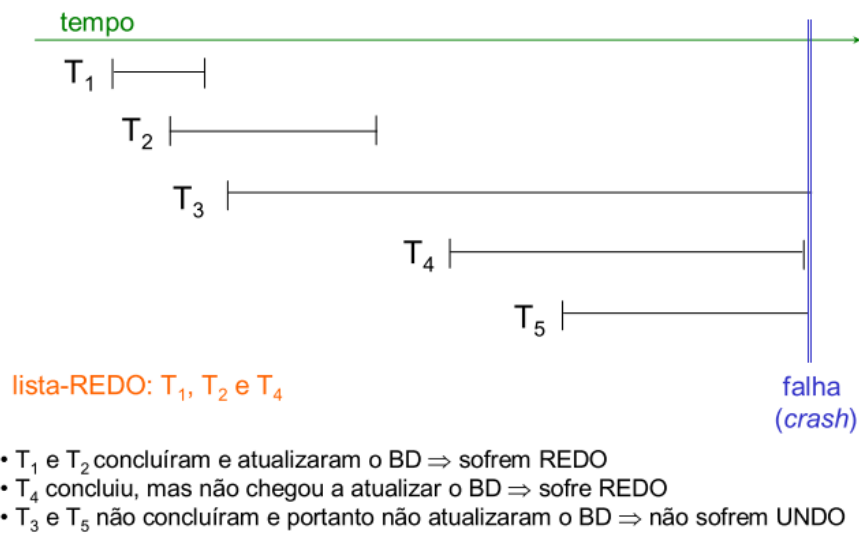


Modificação Postergada do BD (NOT-STEAL / NOT-FORCE)

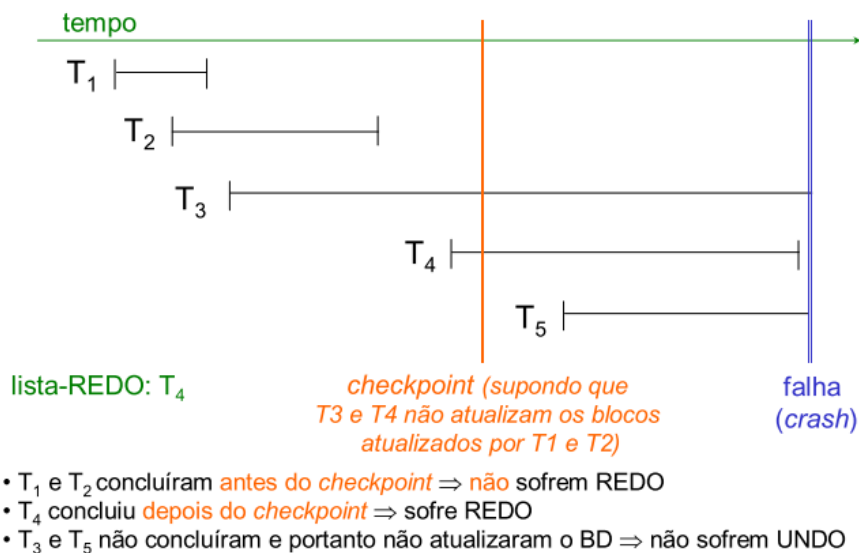
As atualizações de uma transação só podem ir para o disco após o seu commit.

- **Técnica NO-UNDO/REDO:**
 - **Procedimento:** Como nenhuma alteração de transações inacabadas vai para o disco, não há necessidade de **UNDO**. Apenas as transações com **<commit>** no log precisam sofrer **REDO** para garantir que suas atualizações sejam gravadas.

Técnica NO-UNDO/REDO - Exemplo



NO-UNDO/REDO c/ Checkpoint



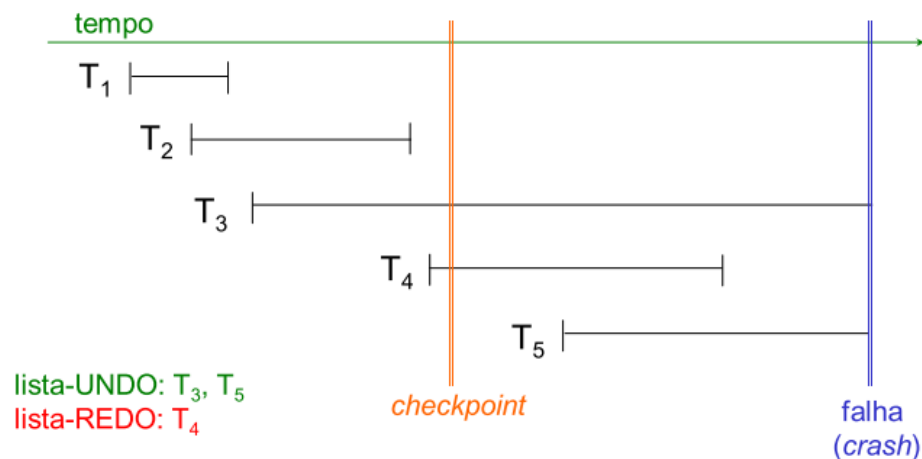
Checkpoints

Para otimizar a recuperação em sistemas com logs muito grandes, utiliza-se a técnica de **Checkpoint**.

- **O que é:** Um ponto no tempo em que o SGBD garante que todas as atualizações de transações já concluídas foram gravadas no disco. Um registro `<checkpoint T1, T2...>` é inserido no log, listando as transações que estavam ativas naquele momento.

- **Vantagem:** Durante a recuperação, o SGBD só precisa analisar o log a partir do último checkpoint. Transações que fizeram commit *antes* do checkpoint não precisam de **REDO**, pois suas alterações já estão garantidamente no disco.

Técnica UNDO/REDO c/ Checkpoint



- T₁ e T₂ concluíram e estão garantidamente no BD ⇒ não sofrem REDO
- T₄ concluiu, mas suas atualizações não necessariamente estão no BD (supondo NOT-FORCE) ⇒ sofre REDO
- T₃ e T₅ não concluíram ⇒ sofrem UNDO

Técnica UNDO/REDO c/ Checkpoint

- Procedimento de *Recovery* – **Passo 1**

Percorre-se o *Log backward* até alcançar um registro *Checkpoint*

- se achou <commit T_x>, insere T_x na **lista-REDO**
- se achou <start T_x> e T_x não está na lista-REDO, insere T_x na **lista-UNDO**

Exemplo de *Recovery* com *Checkpoint*

```

<start T1>
<write T1,A,5,10>
<start T2>
<write T2,C,30,45>
<write T2,E,7,17>
<commit T2>
<write T1,C,45,55>
<start T3>
<write T3,B,15,20>
<commit T1>
<start T4>
<write T4,C,55,65>
<checkpoint T3,T4>
<start T5>
<write T5,D,31,39>
<start T6>
<write T3,A,5,25>
<write T6,F,1,2>
<write T3,E,17,28>
<commit T3>
<write T6,A,25,32>
<start T7>
<write T7,B,20,30>
<commit T7>
<write T4,E,28,34>
Crash!

```

Lista UNDO:

Lista REDO:

Situação no BD
após a falha:

A = 25
B = 20
C = 65
D = 39
E = 17
F = 2

(não necessariamente todos os
writes foram persistidos no BD)

Desejável:

A = 25
B = 30
C = 55
D = 31
E = 28
F = 1

Log

Após a execução do Passo 1

```

<start T1>
<write T1,A,5,10>
<start T2>
<write T2,C,30,45>
<write T2,E,7,17>
<commit T2>
<write T1,C,45,55>
<start T3>
<write T3,B,15,20>
<commit T1>
<start T4>
<write T4,C,55,65>
<checkpoint T3,T4>
<start T5>
<write T5,D,31,39>
<start T6>
<write T3,A,5,25>
<write T6,F,1,2>
<write T3,E,17,28>
<commit T3>
<write T6,A,25,32>
<start T7>
<write T7,B,20,30>
<commit T7>
<write T4,E,28,34>
Crash!

```

Lista UNDO: **T6, T5**

Lista REDO: **T7, T3**

Situação no BD
após a falha:

A = 25
B = 20
C = 65
D = 39
E = 17
F = 2

Desejável:

A = 25
B = 30
C = 55
D = 31
E = 28
F = 1

Log

Técnica UNDO/REDO c/ Checkpoint

- Procedimento de *Recovery* – Passo 2

Analisa-se cada transação T_x no registro *checkpoint*

- se T_x não estiver na lista-REDO, insere T_x na lista-UNDO

```
<start T1>
<write T1,A,5,10>
<start T2>
<write T2,C,30,45>
<write T2,E,7,17>
<commit T2>
<write T1,C,45,55>
<start T3>
<write T3,B,15,20>
<commit T1>
<start T4>
<write T4,C,55,65>
<checkpoint T3,T4>
<start T5>
<write T5,D,31,39>
<start T6>
<write T3,A,5,25>
<write T6,F,1,2>
<write T3,E,17,28>
<commit T3>
<write T6,A,25,32>
<start T7>
<write T7,B,20,30>
<commit T7>
<write T4,E,28,34>
Crash!
```

Após a execução do Passo 2

Lista UNDO: T6, T5, **T4**

Lista REDO: T7, T3

→ neste ponto já se conhece todas as transações que devem sofrer UNDO ou REDO !

Situação no BD
após a falha:

A = 25
B = 20
C = 65
D = 39
E = 17
F = 2

Desejável:

A = 25
B = 30
C = 55
D = 31
E = 28
F = 1

Log

Técnica UNDO/REDO c/ Checkpoint

- Procedimento de *Recovery* – **Passo 3**

Percorre-se de novo o *Log backward*, até que todas as transações em lista-UNDO tenham sofrido UNDO

- marca-se na lista-REDO as transações T_x cujos registros $\langle \text{start } T_x \rangle$ estão sendo encontrados nessa varredura

```
<start T1>
<write T1,A,5,10>
<start T2>
<write T2,C,30,45>
<write T2,E,7,17>
<commit T2>
<write T1,C,45,55>
<start T3>
<write T3,B,15,20>
<commit T1>
<start T4>
<write T4,C,55,65>
<checkpoint T3,T4>
<start T5>
<write T5,D,31,39>
<start T6>
<write T3,A,5,25>
<write T6,F,1,2>
<write T3,E,17,28>
<commit T3>
<write T6,A,25,32>
<start T7>
<write T7,B,20,30>
<commit T7>
<write T4,E,28,34>
Crash!
```

Após a execução do **Passo 3**

Lista UNDO: ~~T6~~, ~~T5~~, ~~T4~~

Lista REDO: T7, T3

Situação no BD
após a falha:

A = 25
B = 20
C = ~~65~~ 55
D = ~~39~~ 31
E = ~~17~~ 28
F = ~~2~~ 1

Desejável:

A = 25
B = 30
C = 55
D = 31
E = 28
F = 1

Log

Técnica UNDO/REDO c/ Checkpoint

- Procedimento de *Recovery* – Passo 4

Caso existam transações não marcadas na lista-REDO ao final da varredura *backward*

- continua-se a varredura *backward* até que todas as transações na lista-REDO tenham sido marcadas

```
<start T1>
<write T1,A,5,10>
<start T2>
<write T2,C,30,45>
<write T2,E,7,17>
<commit T2>
<write T1,C,45,55>
<start T3>
<write T3,B,15,20>
<commit T1>
<start T4>
<write T4,C,55,65>
<checkpoint T3,T4>
<start T5>
<write T5,D,31,39>
<start T6>
<write T3,A,5,25>
<write T6,F,1,2>
<write T3,E,17,28>
<commit T3>
<write T6,A,25,32>
<start T7>
<write T7,B,20,30>
<commit T7>
<write T4,E,28,34>
Crash!
```

Após a execução do Passo 4

Lista UNDO: ~~T6~~, ~~T5~~, ~~T4~~

Lista REDO: T7, T3
✓ ✓

Situação no BD
após a falha:

A = 25
B = 20
C = ~~65~~ 55
D = ~~39~~ 31
E = ~~17~~ 28
F = ~~2~~ 1

Desejável:

A = 25
B = 30
C = 55
D = 31
E = 28
F = 1

Log

Técnica UNDO/REDO c/ Checkpoint

- Procedimento de *Recovery* – Passo 5

Percorre-se o *Log forward* do ponto de parada, realizado REDO das transações na lista-REDO

```
<start T1>
<write T1,A,5,10>
<start T2>
<write T2,C,30,45>
<write T2,E,7,17>
<commit T2>
<write T1,C,45,55>
<start T3>
<write T3,B,15,20>
<commit T1>
<start T4>
<write T4,C,55,65>
<checkpoint T3,T4>
<start T5>
<write T5,D,31,39>
<start T6>
<write T3,A,5,25>
<write T6,F,1,2>
<write T3,E,17,28>
<commit T3>
<write T6,A,25,32>
<start T7>
<write T7,B,20,30>
<commit T7>
<write T4,E,28,34>
Crash!
```

Após a execução do Passo 5

Lista UNDO: ~~T6~~, ~~T5~~, ~~T4~~
Lista REDO: ~~T7~~, ~~T3~~

Situação no BD após a falha:

A = 25	✓
B = 20	✓ 20 30
C = 65	55
D = 39	31
E = 17	28 ✓
F = 2	1

Desejável:
A = 25
B = 30
C = 55
D = 31
E = 28
F = 1

Vantagem da técnica:
Não é necessário varrer todo o Log para realizar o *Recovery*!

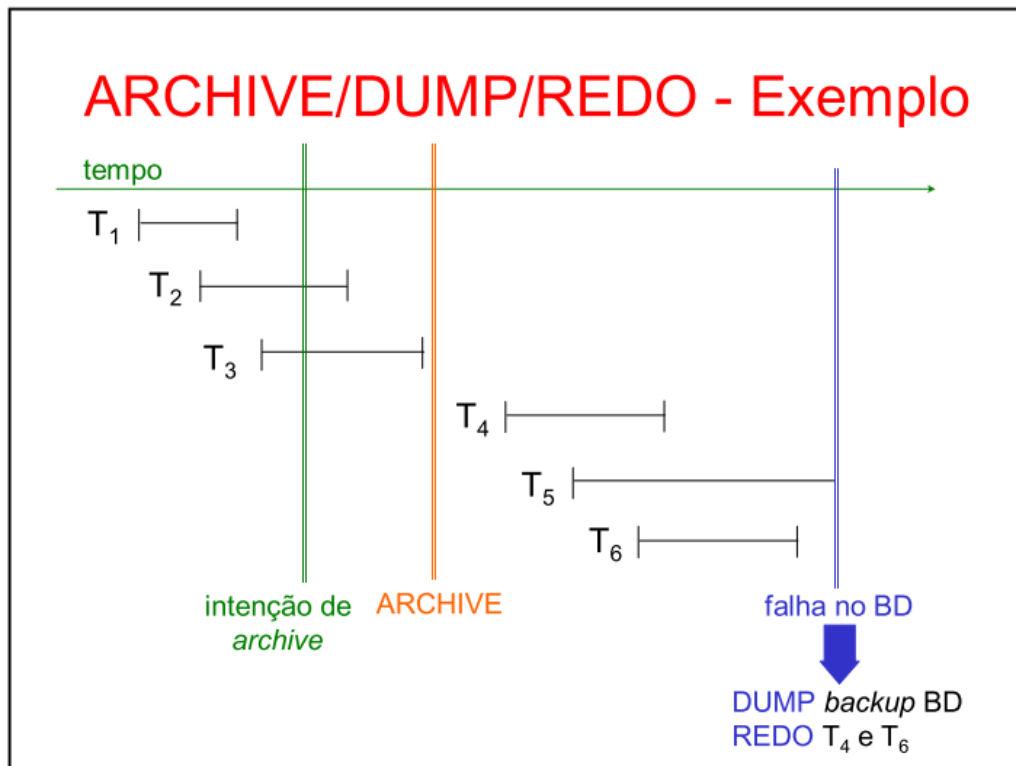
Recuperação de falha de meio de armazenamento

Técnica ARCHIVE/DUMP/REDO: Usada para falhas de disco.

- Procedimento:

1. Restaura-se o banco de dados a partir do último backup (DUMP).

2. Realiza-se uma varredura para frente (*forward*) no log (que também deve ter um backup) para refazer (REDO) todas as transações que foram concluídas desde o último backup.



Técnica de Shadow Pages (NO_UNDO/NO-REDO)

Uma abordagem alternativa que não utiliza log.

- **Como funciona:** Mantém duas tabelas de páginas de disco: a Corrente (TPC) e a Sombra (TPS). A TPS é uma cópia da TPC no início da transação. Quando a transação atualiza uma página, uma nova cópia da página é criada em um local livre do disco, e a TPC é atualizada para apontar para essa nova página. A TPS permanece inalterada.
- **Recuperação:** Se a transação falhar antes do commit, a TPC modificada é simplesmente descartada, e a TPS (que aponta para o estado original dos dados) torna-se a nova TPC. Não são necessários UNDO ou REDO.
- **Desvantagens:** É complexa para ambientes multiusuário, pode causar fragmentação no disco (não mantém dados clusterizados) e requer um processo de "coleta de lixo" para gerenciar páginas obsoletas.

Controle de Concorrência

Por que Controle de Concorrência?

- **Necessidade:** Em SGBDs multiusuário, várias transações podem executar simultaneamente (concorrentemente) para melhorar a eficiência e o uso da CPU (evitando ociosidade durante operações de I/O).
- **Problema:** A execução concorrente não controlada pode levar a problemas de isolamento, resultando em estados inconsistentes do banco de dados. Exemplos clássicos são:
 - **Atualização Perdida (Lost Update):** Uma transação sobrescreve a atualização feita por outra.

T1	T2
read(X)	
$X = X - 20$	
	read(Z)
	$X = Z + 10$
write(X)	
read(Y)	
	write(X)
$Y = X + 30$	
write(Y)	

a atualização de X por T1 foi perdida!

- **Leitura Suja (Dirty Read):** Uma transação lê dados modificados por outra transação que ainda não foi confirmada (e pode ser abortada depois).

T1	T2
read(X)	
$X = X - 20$	
write(X)	
	read(X)
	$Z = X + 10$
	write(Z)
read(Y)	
abort()	

T2 leu um valor de X que não será mais válido!

- **Solução:** O **Scheduler** (ou subsistema de controle de concorrência) organiza a ordem de execução das operações das transações concorrentes (criando um escalonamento ou *schedule*) para garantir o isolamento e a consistência, como se as transações fossem executadas serialmente

Escalonamento e recuperação

Escalonamento Recuperável

- Garante que, se T_x realizou *commit*, T_x não irá sofrer UNDO
 - o *recovery* espera sempre esse tipo de escalonamento!
 - garantia de **Durabilidade!**
- Um escalonamento E é **recuperável** se nenhuma T_x em E for concluída até que todas as transações que gravaram dados lidos por T_x tenham sido concluídas

escalonamento não-recuperável	T1	T2	escalonamento recuperável	T1	T2
	read(X)			read(X)	
	$X = X - 20$			$X = X - 20$	
	write(X)			write(X)	
		read(X)			read(X)
		$X = X + 10$			$X = X + 10$
		write(X)			write(X)
		commit()		commit()	
	abort()				commit()

Escalonamento sem Aborto em Cascata

- Um escalonamento recuperável pode gerar abortos de transações em cascata
 - consome muito tempo de *recovery*!
- Um escalonamento E é **recuperável e evita aborto em cascata** se uma T_x em E só puder ler dados que tenham sido atualizados por transações que já concluíram
 - Evita **dirty-read**!

escalonamento recuperável com aborto em cascata	T1	T2	escalonamento recuperável sem aborto em cascata	T1	T2
	read(X)			read(X)	
	$X = X - 20$			$X = X - 20$	
	write(X)			write(X)	
		read(X)		commit()	
		$X = X + 10$			read(X)
		write(X)			$X = X + 10$
	abort()	...			write(X)
					...

Escalonamento Estrito

- Garante que, se T_x deve sofrer UNDO, basta gravar a *before image* dos dados atualizados por ela
 - $UNDO(T_x)$ não interferirá em atualizações posteriores de outras transações
 - Evita *lost-update*!
- Um escalonamento E é recuperável, evita aborto em cascata e é estrito se uma T_x em E só puder ler ou atualizar um dado X depois que todas as transações que atualizaram X tenham sido concluídas

escalonamento recuperável sem aborto em cascata e não-estricto	T1	T2	escalonamento recuperável sem aborto em cascata e estrito	T1	T2
	read(X)			read(X)	
	$X = X - 20$			$X = X - 20$	
	write(X)			write(X)	
		read(Y)		commit()	
		$X = Y + 10$			read(Y)
		write(X)			$X = Y + 10$
		commit()			write(X)
	abort()				commit()

Serializabilidade

- **Conceito:** Um escalonamento concorrente é considerado correto (válido) se for serializável, ou seja, se produzir o mesmo resultado final e tiver o mesmo efeito no banco de dados que *alguma* execução serial (uma após a outra) das mesmas transações.
- **Verificação (Teoria):**
 - **Serializabilidade por Conflito:** Um escalonamento é serializável por conflito se a ordem de quaisquer duas operações conflitantes (de transações diferentes, no mesmo dado, com pelo menos uma escrita) for a mesma que em algum escalonamento serial. Pode ser verificada construindo um **grafo de precedência**: se o grafo não tiver ciclos, o escalonamento é serializável por conflito.
 - **Serializabilidade por Visão:** Uma definição mais geral, mas mais complexa de verificar. Garante que cada leitura veja o mesmo valor e que a escrita final em cada dado seja da mesma transação que em algum escalonamento serial. Todo escalonamento serializável por conflito também é serializável por visão, mas o contrário não é sempre verdade.
- **Prática:** Testar serializabilidade diretamente é impraticável em sistemas reais devido à dinâmica das transações. Por isso, usam-se protocolos que *garantem* a serializabilidade.

Técnicas de Controle de Concorrência

Classificadas como Pessimistas ou Otimistas.

Técnicas Pessimistas (assumem conflitos frequentes) → Supõem que sempre ocorre interferência entre transações e garantem a serializabilidade enquanto a transação está ativa

- **Baseadas em Bloqueio (Locking):** Mais comuns. Transações solicitam bloqueios nos dados antes de acessá-los.
- **Modos de Bloqueio:**

- **Compartilhado (Shared - S):** Para leitura. Várias transações podem ter um lock S no mesmo dado simultaneamente.
- **Exclusivo (Exclusive - X):** Para escrita (ou leitura+escrita). Apenas uma transação pode ter um lock X em um dado; nenhum outro lock (nem S nem X) é permitido.

• Operações

- $ls(D)$: solicitação de bloqueio compartilhado sobre D
- $lx(D)$: solicitação de bloqueio exclusivo sobre D
- $u(D)$: libera o bloqueio atual sobre D

Exemplo de História com Bloqueios

T1	T2
$lock-S(Y)$	
$read(Y)$	
$unlock(Y)$	
	$lock-S(X)$
	$lock-X(Y)$
	$read(X)$
	$read(Y)$
	$unlock(X)$
	$write(Y)$
	$unlock(Y)$
	$commit()$
$lock-X(X)$	
$read(X)$	
$write(X)$	
$unlock(X)$	
$commit()$	

$H = ls1(Y) \ r1(Y) \ u1(Y) \ ls2(X) \ lx2(Y)$
 $\quad r2(X) \ r2(Y) \ u2(X) \ w2(Y) \ u2(Y) \ c2$
 $\quad lx1(X) \ r1(X) \ w1(X) \ u1(X) \ c1$

- **Protocolo de Bloqueio de Duas Fases (Two-Phase Locking - 2PL):** Garante a serializabilidade. Toda transação passa por duas fases:
 1. **Fase de Crescimento (Expansão):** A transação pode adquirir bloqueios, mas não pode liberar nenhum.
 2. **Fase de Encolhimento (Retrocesso):** A transação pode liberar bloqueios, mas não pode adquirir nenhum novo.
- **Deadlock (Impasse):** Ocorre quando duas ou mais transações ficam bloqueadas esperando umas pelas outras para liberar locks.
- **Deteção:** Usando um grafo de espera; um ciclo indica deadlock. Uma transação "vítima" é escolhida para ser abortada.
- **Baseadas em Timestamp:**

Técnicas Baseadas em *Timestamp*

- *Timestamp*
 - rótulo de tempo associado à T_x ($TS(T_x)$)
- Técnicas
 - consideram que T_x deseja um dado bloqueado por outra transação T_y
 - Técnica 1: esperar-ou-morrer (*wait-die*)
 - se $TS(T_x) < TS(T_y)$ então T_x espera
senão início
 $abort(T_x)$
 $start(T_x)$ com o mesmo TS
fim

Técnicas Baseadas em *Timestamp*

- *Timestamp*
 - rótulo de tempo associado à T_x ($TS(T_x)$)
- Técnicas
 - consideram que T_x deseja um dado bloqueado por outra transação T_y
 - Técnica 1: esperar-ou-morrer (*wait-die*)
 - se $TS(T_x) < TS(T_y)$ então T_x espera
senão início
 $abort(T_x)$
 $start(T_x)$ com o mesmo TS
fim

Técnica *Cautious-Waiting*

- **Princípio de Funcionamento**
 - se T_x deseja D e D está bloqueado por T_y
então
 se T_y não está em alguma Fila-WAIT
 então T_x espera
 senão início
 $abort(T_x)$
 $start(T_x)$
 fim
- **Vantagem**
 - se T_y já está em espera, T_x é abortada para evitar um possível ciclo de espera
- **Desvantagem**
 - a mesma das técnicas baseadas em *timestamp*

Técnicas Otimistas (assumem conflitos raros)

- supõem que quase nunca ocorre interferência entre transações e verificam a serializabilidade somente ao final de uma transação
 - **Baseadas em Validação:** Transações executam livremente em cópias locais dos dados. Antes do commit, passam por uma fase de **validação** para verificar se houve conflito com outras transações que já foram validadas ou estão em validação. Se a validação falhar, a transação é abortada ; se for bem-sucedida, suas alterações (das cópias locais) são aplicadas ao BD.
 - **Fases:** Leitura, Validação, Escrita.
 - **Validação:** Usa timestamps (Start, Validation, Finish) e conjuntos de leitura/escrita (Read Set, Write Set) para checar condições específicas que garantem a serializabilidade.

Indexação

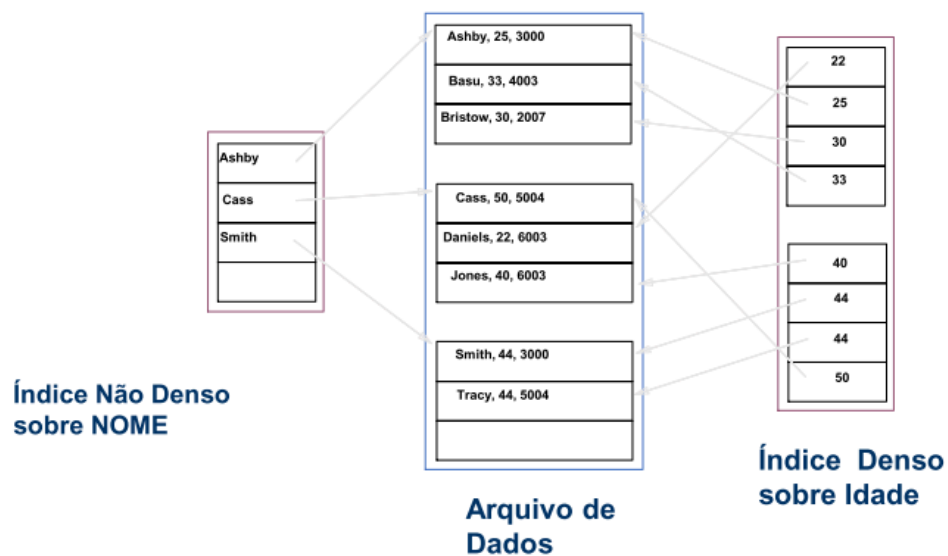
O que são índices?

- São estruturas de dados adicionais que armazenam informações sobre os registros de dados de uma tabela
- O objetivo é acelerar a recuperação de registros baseada nos valores de um ou mais campos específicos, chamados campos de indexação ou chave de busca
- Em essência, um índice funciona como um “guia” que aponta para a localização dos registros, permitindo encontrá-los mais rapidamente

Tipos de índices

- **Índice Primário:** Construído sobre o campo-chave usado para ordenar fisicamente os registros no arquivo (campo de classificação). Esse campo deve ter valores únicos. Só pode haver um índice primário (ou clustering) por arquivo.

- **Índice Clustering (Agrupamento):** Construído sobre o campo usado para ordenar fisicamente os registros, mas esse campo não é chave (permite valores repetidos). Também só pode haver um por arquivo.
- **Índice Secundário:** Construído sobre qualquer campo que não seja usado para a ordenação física dos dados. Pode haver vários índices secundários para um mesmo arquivo. Pode ser sobre um campo chave-candidata (valores únicos) ou sobre um campo não-chave (valores repetidos).
- **Índice Denso:** Possui uma entrada no índice para cada registro no arquivo de dados. Os registros no arquivo de dados podem estar em qualquer ordem. Índices secundários são geralmente densos.
- **Índice Esparso (Não Denso):** Possui uma entrada no índice apenas para alguns registros, tipicamente um por bloco ou página do arquivo de dados. Requer que o arquivo de dados esteja ordenado pelo campo de indexação. Índices primários e clustering são esparsos.



	Campo de Indexação Utilizado para Classificar o Arquivo	Campo de Indexação Não Utilizado para Classificar o Arquivo
Campo de indexação é chave (não admite repetição de valor)	Índice Primário	Índice Secundário (sobre chave candidata)
Campo de indexação não é chave (admite repetição de valor)	Índice Clustering	Índice Secundário (sobre qualquer atributo não chave)

Tipo de Índice	Número de Entradas de Índice (Primeiro Nível)	Denso ou Esperso	Âncora de Bloco no Arquivo de Dados
Primário	Número de blocos do arquivo de dados	Esperso	Sim
Clustering	Número de valores distintos do campo de indexação	Esperso	Sim/Não (*)
Secundário (sobre Chave Candidata)	Número de registros do arquivo de dados	Denso	Não
Secundário (sobre campo que não é chave)	Número de registros ou número de valores distintos do campo de indexação	Denso ou Esperso	Não

Estrutura de índices comuns

- **Índices de Múltiplos Níveis:** Como um índice (de nível único) é um arquivo ordenado, pode-se criar um índice (esperso) sobre ele, criando uma estrutura de múltiplos níveis. Isso reduz ainda mais o número de acessos necessários para a busca.
- **Árvores de Busca:** Estruturas hierárquicas usadas para organizar os índices. Cada nó contém valores de chave e ponteiros para nós filhos ou para os dados.
- **Árvores B (B-Trees):** Estruturas de árvore balanceadas e dinâmicas, muito usadas para índices. Mantêm-se balanceadas automaticamente durante inserções e deleções através de divisões e junções de nós. Todos os nós folha estão na mesma profundidade. Eficientes para buscas e travessias. Nós internos contêm chaves e ponteiros para nós filhos e ponteiros para os dados.
- **Árvores B+ (B+-Trees):** Variação popular da Árvore B e a mais comum em SGBDs. Diferenças principais:
 - Nós internos contêm apenas chaves e ponteiros para outros nós (filhos).
 - Ponteiros para os registros de dados só existem nos nós folha.
 - Os nós folha são ligados sequencialmente (como uma lista ligada), otimizando varreduras em ordem (range scans).

Índices no PostgreSQL

- **Tipos Disponíveis:** B-tree (padrão), Hash, GiST, GIN. (R-tree é obsoleto e foi substituído por GiST).
 - **B-tree:** É o tipo padrão (`CREATE INDEX` sem `USING`). Ideal para a maioria dos casos, suportando consultas de igualdade (`=`) e faixa (`<` , `>` , `<=` , `>=`) em dados ordenáveis. Também útil para `LIKE` (com padrões prefixados).

```
CREATE INDEX nome ON tabela USING BTREE(coluna)
```

- **Hash:** Indicado apenas para comparações de igualdade simples (`=`). Seu uso é desencorajado, pois o B-tree geralmente oferece desempenho similar ou melhor e tem outras vantagens.

```
CREATE INDEX nome ON tabela USING HASH(coluna)
```

- **GiST (Generalized Search Tree):** Uma infraestrutura flexível para implementar diferentes estratégias de indexação. Muito usado para dados geométricos/espaciais e busca textual.
- **GIN (Generalized Inverted Index):** Índice invertido, adequado para indexar valores que contêm múltiplos componentes, como **arrays ou documentos para busca textual**. Suporta operadores como `contém`, `está contido em`, `sobreposição` (`@>`, `<@`, `&&`) para arrays

Exemplos

- Criando um índice

```
CREATE INDEX idx_pessoas_nome ON pessoas USING hash (nome);
```

- Criando um índice que incorpora dois campos ao mesmo tempo

```
CREATE INDEX idx_usuario_login_senha ON usuario USING btree (login, senha);
```

- Criando um índice, mas indexando apenas parte do campo

```
CREATE INDEX idx_pessoas_endereco ON pessoas  
USING btree (substr(endereco, 1, 10));
```

- Criando um índice em um determinado tablespace

```
CREATE INDEX idx_compras ON compras (item) TABLESPACE outro_tablespace;
```

- Criando um índice com a cláusula WHERE

```
CREATE UNIQUE INDEX ind_nome_tiago ON pessoa USING GIN (nome)  
WHERE nome ILIKE 'Tiago%';
```

- Renomeando um índice

```
CREATE INDEX idx_pessoas_endereco RENAME TO idx_pessoas_endereco_completo;
```

- Excluindo um índice

```
DROP INDEX idx_pessoas_endereco_completo
```

- **UNIQUE** : Garante que não haja valores duplicados nas colunas indexadas.
- **CONCURRENTLY** : Permite criar o índice sem bloquear escritas na tabela (útil para tabelas grandes em produção)